

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

o0o



XỬ LÝ NGÔN NGỮ TỰ NHIÊN

Đề tài: Hệ thống RAG đề xuất Tai nghe

Lớp : Xử lý ngôn ngữ tự nhiên - N02

Số thứ tự nhóm: 14

Đào Văn Công MSSV: B22DCKH010

Nguyễn Văn Quý MSSV: B22DCKH100

Đào Thịnh Phú MSSV: B22DCKH086

Link Github: <https://github.com/DeepLearnRecomSysDataMining/NLP-RAG-HeadphoneData>

HÀ NỘI, 05/2026

CHƯƠNG 1. GIỚI THIỆU VÀ TẬP DỮ LIỆU

1.1 Đặt vấn đề

Thị trường thiết bị âm thanh tại Việt Nam ngày càng phong phú với hàng trăm thương hiệu quốc tế và hàng nghìn mẫu sản phẩm thuộc mọi phân khúc giá. Người tiêu dùng đối mặt với ba thách thức chính khi tìm kiếm: Thuật ngữ kỹ thuật; Ngôn ngữ truy vấn tự nhiên tiếng Việt không nhất quán; Công cụ tìm kiếm truyền thống dựa trên từ khóa không nắm bắt được ngữ nghĩa sâu.

Kiến trúc Retrieval-Augmented Generation (RAG) đã chứng minh hiệu quả trong các bài toán Q-A, kết hợp truy xuất thông tin chính xác với khả năng sinh ngôn ngữ tự nhiên. Tuy nhiên, ứng dụng RAG cho tiếng Việt trong lĩnh vực thương mại điện tử còn rất hạn chế.

Đề án này xây dựng hệ thống RAG chuyên biệt với các đóng góp chính:

- Pipeline kết hợp tiền xử lý, chunking và xây dựng embedding vector.
- Hybrid Search tích hợp tokenizer PyVi.
- Cơ chế Metadata-aware Reranking với bonus/penalty ngữ cảnh.
- bộ đánh giá query với 4 chỉ số chuẩn Information Retrieval.

1.2 Cơ sở lý thuyết

Retrieval-Augmented Generation: kiến trúc RAG, cách chunking hợp lý để đảm bảo truy vấn đạt kết quả tốt nhất.

Hybrid Search: chứng minh rằng sparse (BM25) và dense retrieval bổ sung cho nhau: BM25 mạnh về exact-match từ khóa và dense search mạnh về truy vấn ngữ nghĩa. Kết hợp hai cách cải thiện chất lượng hơn.

Reranking với Cross-Encoder: dùng BERT làm cross-encoder để rerank: cho query và passage qua cơ chế attention chung, tính điểm liên quan chính xác hơn bi-encoder độc lập. BGE-Reranker mở rộng cách tiếp cận này cho đa ngôn ngữ - Multilingual.

Xử lý tiếng Việt: tokenizer xử lý từ ghép tiếng Việt. Vietnamese Bi-Encoder của BKA1 được fine-tune trên corpus tiếng Việt, phù hợp cho bài toán semantic search trong nước.

1.3 Thu thập Dữ liệu

1.3.1 Nguồn dữ liệu và Crawler

Dữ liệu được thu thập từ tainghe.com.vn – trang thương mại điện tử chuyên ngành tai nghe lớn tại Việt Nam, cung cấp bài đánh giá chuyên sâu cho từng sản phẩm.

Crawler được thiết kế theo kiến trúc 3 bước:

Bước 1 – Phát hiện danh mục: Quét trang chủ, lọc các URL chứa từ khóa liên quan (tai-nghe, dac-amp, loa, hi-end-cables).

Bước 2 – Thu thập link sản phẩm: Duyệt qua từng trang phân trang của mỗi danh mục, trích xuất các thông tin.

Bước 3 – Bóc tách chi tiết: Sử dụng `ThreadPoolExecutor (max_workers=4)` xử lý song song. Thuật toán *Bottom-up scanning* trích xuất thương hiệu: duyệt tất cả thẻ HTML chứa “thương hiệu”, sắp xếp theo độ dài tăng dần và trích tên hãng chính xác.

1.3.2 Đặc điểm Tập dữ liệu

Bảng 1.1: Thống kê tập dữ liệu thu thập từ tainghe.com.vn

Tổng sản phẩm	> 2300 sản phẩm
Số thương hiệu nhận diện	40+ (Sony, JBL, ThieAudio, PWAudio, FiiO...)
Các cột dữ liệu	url, product_name, brand, price, review_content
Ngôn ngữ	Tiếng Việt + thuật ngữ kỹ thuật tiếng Anh
Chiều dài review-content	300–3000 ký tự/sản phẩm
Định dạng lưu trữ	CSV

1.4 Tiền xử lý Dữ liệu

Hàm `prepare_data()` đảm nhiệm vai trò chuẩn bị và làm sạch dữ liệu trước khi đưa vào không gian vector. Quá trình này được thực hiện qua các bước kỹ thuật sau:

Làm sạch và trích xuất đặc trưng: Hệ thống kiểm tra sự tồn tại của các trường dữ

liệu bắt buộc (`product_name`, `review_content`, `price`, `url`). Các dòng bị thiếu tên sản phẩm hoặc nội dung đánh giá (NaN) sẽ bị loại bỏ hoàn toàn.

- **Giá cả:** Được xử lý qua hàm `clean_price()` để loại bỏ các ký tự thừa và chuyển về định dạng số nguyên (integer).
- **Thương hiệu (Brand):** Hệ thống ưu tiên sử dụng dữ liệu từ cột `brand` có sẵn. Nếu giá trị bị thiếu hoặc không hợp lệ (như NaN, none, khác), hệ thống sẽ tự động dùng hàm `extract_brand()` để quét và trích xuất tên hãng trực tiếp từ chuỗi `product_name` dựa trên từ điển từ khóa.

Nhúng Metadata (Metadata Injection): Trước khi embedding, mỗi sản phẩm được gộp thành một chuỗi văn bản (string) duy nhất chứa toàn bộ ngữ cảnh. Công thức ghép chuỗi như sau:

`c = "Sản phẩm: " + n + ". Thương hiệu: " + b + ". Giá: " + p + ". " + r`

trong đó *n, b, p, r* lần lượt là tên, thương hiệu, giá và nội dung bài review gốc.

Việc đóng gói cấu trúc này giúp cả mô hình Dense (FAISS) lẫn Sparse (BM25) đều có thể đối sánh chính xác thông tin trọng tâm ngay trong nội dung văn bản.

Các thông tin gốc và định dạng chuẩn của URL (`normalize_url`) cũng được lưu song song vào metadata của từng Document.

3. Chunking (Adaptive Chunking) và Chuẩn hóa: Để tối ưu hóa không gian vector, hệ thống không cắt nhỏ toàn bộ dữ liệu mà áp dụng chiến lược thích ứng:

- Nếu bài review dài vượt ngưỡng cấu hình (`CHUNK_IF_LONGER_THAN`), hệ thống dùng `RecursiveCharacterTextSplitter` để cắt nhỏ văn bản thành các chunk (với `chunk_size` và `chunk_overlap` xác định).
- Nếu bài review ngắn, nó được giữ nguyên vẹn thành một Document duy nhất để bảo toàn tuyệt đối ngữ nghĩa gốc.

Cuối cùng, tất cả văn bản (hoặc chunk) đều qua hàm `clean_vietnamese_text()` để loại bỏ ký tự đặc biệt, đưa về chữ thường và phân từ (tokenize) bằng tokenizer `ViTokenizer` của PyVi.

CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG

2.1 Tổng quan Pipeline

Hệ thống được xây dựng theo pipeline RAG 5 giai đoạn kết nối liền mạch từ thu thập dữ liệu đến trả kết quả, minh họa tại Hình 2.1.



Hình 2.1: Pipeline RAG 5 giai đoạn của hệ thống

1. Nhóm Thu thập dữ liệu

- `get_all_subcategories()`: Quét trang chủ để tìm và lọc ra các đường dẫn (URL) của các danh mục sản phẩm liên quan (như tai nghe, loa, dac-amp).
- `get_product_links_from_category()`: Duyệt qua các trang phân trang (pagination) của từng danh mục để gom toàn bộ link chi tiết của các sản phẩm.
- `fetch_product_details()`: Truy cập trực tiếp vào trang chi tiết sản phẩm, sử dụng kỹ thuật phân tích cú pháp HTML (qua thư viện BeautifulSoup) để trích xuất các trường dữ liệu quan trọng như Tên, Giá, Thương hiệu và Nội dung bài đánh giá.
- `run_full_scraper()`: Hàm điều phối chính (entry point) của luồng thu thập dữ liệu, áp dụng kỹ thuật xử lý đa luồng (ThreadPoolExecutor) để tối ưu tốc độ cào dữ liệu và tự động lưu kết quả đầu ra dưới định dạng CSV.

2. Nhóm Tiền xử lý & Tiện ích (Utils)

- `clean_vietnamese_text()`: Chuẩn hoá và phân từ (tokenize) tiếng Việt bằng thư viện PyVi.
- `parse_price_constraint()`: Trích xuất các điều kiện về giá (ví dụ: “dưới 2 triệu”, “tầm 500k”) từ câu truy vấn tự nhiên để làm tiền đề cho bộ lọc cứng (Hard Filter).
- `extract_brand()`: Trích xuất và nhận diện tên thương hiệu từ metadata của sản phẩm hoặc từ câu truy vấn của người dùng.

- `dedup_by_url()`: Loại bỏ các Document trùng lặp cùng một sản phẩm (dựa trên khóa duy nhất là URL).

3. Nhóm Xây dựng Dữ liệu (Data Pipeline)

- `prepare_data()`: Đọc file CSV, làm sạch dữ liệu, áp dụng kỹ thuật Adaptive Chunking và đóng gói thành các đối tượng Document của LangChain.
- `build_index()`: Tính toán các Vector Embedding thông qua SentenceTransformer và lưu trữ cục bộ vào cơ sở dữ liệu vector FAISS.
- `load_index()`: Tải cấu trúc FAISS index, khởi tạo mô hình BM25 và Cross-Encoder vào bộ nhớ để sẵn sàng phục vụ quá trình suy luận.

4. Nhóm Truy xuất & Xếp hạng (Retrieval & Reranking)

- `retrieve_dense()`: Tìm kiếm vector (Dense Search) để tìm các tài liệu liên quan nhất theo cosine similarity từ FAISS.
- `retrieve_hybrid()`: Hợp nhất các kết quả từ hai luồng Dense Search và Sparse Search (BM25), sau đó deduplicate để tạo pool ứng viên chất lượng cao.
- `rerank_candidates()`: Chấm điểm lại các ứng viên bằng mô hình Cross-Encoder, cộng điểm Brand Bonus, loại bỏ ứng viên sai bằng Hard Price Filter.
- `rag_answer()`: Đóng vai trò là điểm vào (entry point) chính, tiếp nhận kết quả từ bước xếp hạng và định dạng lại ngữ cảnh (context) để sẵn sàng trả lời.

2.2 Mô hình Nhúng và Vector Store

Hệ thống sử dụng mô hình `bkai-foundation-models/vietnamese-bi-encoder` để nhúng văn bản thành vector $\mathbf{v} \in \mathbb{R}^{768}$. Đây là mô hình Bi-Encoder được fine-tune đặc biệt trên corpus tiếng Việt, đảm bảo chất lượng biểu diễn ngữ nghĩa cho các thuật ngữ âm thanh học chuyên ngành.

Tìm kiếm k ứng viên gần nhất trong không gian vector bằng cosine similarity:

$$\text{sim}(q, d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|} \quad (2.1)$$

Vector store FAISS dùng để lưu trữ và truy xuất vector. Để đảm bảo tốc độ tối đa khi encode, SentenceTransformer được thiết lập nhận diện phần cứng

GPU/CUDA hay là CPU để tự động chỉnh `batch_size`, sau đó kết quả được đóng gói vào FAISS thông qua LangChain.

Hệ thống áp dụng chiến lược **Adaptive Chunking**: Chỉ những bài đánh giá dài (trên 1000 ký tự) mới bị chia nhỏ (`chunk_size = 500`, `overlap = 50`) bằng `RecursiveCharacterTextSplitter`. Các đánh giá ngắn được giữ nguyên thành một `Document` để giữ nguyên ngữ nghĩa gốc.

2.3 Hybrid Retrieval

2.3.1 Tiền xử lý và Nhúng Metadata (Metadata Injection)

Trước khi đưa vào lưu trữ và tìm kiếm, mỗi tài liệu được hệ thống chuẩn hoá thành một chuỗi dạng: “*Sản phẩm: [Tên]. Thương hiệu: [Hãng]. Giá: [Giá]. [Nội dung Review]*”. Nhúng trực tiếp metadata vào giúp cả mô hình Dense và Sparse dễ dàng bắt được các từ khóa trọng tâm.

2.3.2 Dense Search

Với mỗi câu truy vấn q , hệ thống lấy $k_d = 25$ tài liệu gần nhất từ FAISS theo công thức (??). Query được tiền xử lý qua `clean_vietnamese_text()` trước khi nhúng, đồng bộ với xử lý khi build index.

2.3.3 Sparse Search (BM25)

BM25 tính điểm liên quan:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{dl}\right)} \quad (2.2)$$

với $k_1 = 1.5$, $b = 0.75$ (mặc định Okapi BM25). Hệ thống trích xuất $k_s = 25$ tài liệu có điểm cao nhất. Cả corpus lẫn query đều được tokenize bằng PyVi, đảm bảo giống về cách tách từ tiếng Việt giúp so khớp chính xác hơn.

2.3.4 Fusion và Deduplication

Kết quả $k_d = 25$ từ Dense và $k_s = 25$ từ BM25 được hợp nhất và **deduplicate theo URL** (khóa duy nhất cho mỗi sản phẩm), tạo pool ứng viên gồm khoảng 25–50 tài liệu chất lượng cao cho bước Reranking.

2.4 Cross-Encoder Reranking và Metadata Filtering

Mô hình BAAI/bge-reranker-v2-m3 chấm điểm từng cặp (query, document) bằng cơ chế cross-attention toàn diện, chính xác hơn bi-encoder nhưng tốn kém hơn

về tính toán. Danh sách ứng viên và điểm số sau đó được tinh chỉnh dựa trên ngữ cảnh metadata:

- **Cộng điểm thương hiệu (Brand Bonus):**

$$s_{\text{final}}(q, d) = \text{CE}(q, d) + \Delta_{\text{brand}} \quad (2.3)$$

với $\Delta_{\text{brand}} = +0.5$ nếu thương hiệu trong metadata khớp với các thương hiệu được phát hiện trong câu truy vấn q .

- **Lọc cứng theo giá (Hard Filter):** Hệ thống phân tích điều kiện giá từ query (ví dụ: “dưới 1 triệu”, “tầm 500k”). Nếu giá của sản phẩm d không thỏa mãn ngưỡng yêu cầu, ứng viên đó sẽ bị loại bỏ hoàn toàn khỏi danh sách kết quả thay vì sử dụng cơ chế trừ điểm.

Sau khi tính điểm s_{final} và lọc cứng các điều kiện, hệ thống tiến hành deduplicate lần 2 theo URL và trả về Top-5 kết quả cuối cùng phục vụ cho LLM sinh câu trả lời.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1 Cài đặt Thực nghiệm

Môi trường và Tham số cấu hình

EMBED_MODEL_NAME : vietnamese-bi-encoder (BKAI)

RERANK_MODEL_NAME : bge-reranker-v2-m3 (BAAI)

Embedding dim : 768

Batch size encode : 256 (GPU) / 32 (CPU)

TOP_K_RETRIEVE : 25

TOP_K_FINAL : 5

CHUNK_SIZE : 500 ký tự

CHUNK_OVERLAP : 50 ký tự

CHUNK_IF_LONGER_THAN : 1000 ký tự

3.2 Bộ Câu hỏi Đánh giá

Bộ test gồm 12 truy vấn được thiết kế bao phủ các tình huống thực tế đa dạng mà người dùng thường gặp khi tìm kiếm sản phẩm tai nghe.

- **Exact match tên sản phẩm đầy đủ:** kiểm tra khả năng bắt từ khóa chính xác của *BM25 keyword*.
- **Brand + điều kiện giá:** kiểm tra *Metadata Filter* (lọc cứng theo giá và hãng).
- **Tìm theo dòng sản phẩm (Series):** kiểm tra khả năng tìm kiếm ngữ nghĩa của *Vector* và độ chính xác của *Rerank*.
- **Sai chính tả nhẹ (Ignit vs Ignis):** kiểm tra độ bền bỉ (*robustness*) của mô hình *Embedding*.
- **Tính năng kỹ thuật cụ thể:** kiểm tra khả năng hiểu ngữ cảnh chuyên sâu của bước *Rerank*.
- **Truy vấn ngữ nghĩa mở rộng:** kiểm tra khả năng tìm kiếm dựa trên ý định (*Dense semantic*).
- **Exact match tên + model number:** kiểm tra sự kết hợp hiệu quả giữa *BM25* và *Vector*.
- **Loại + brand + giá kết hợp:** kiểm tra tổng hợp khả năng phân tích và lọc *Metadata Filter*.
- **Thông số vật liệu (OCC, 4.4mm):** kiểm tra khả năng bắt các thông số kỹ thuật qua *Semantic search*.

- **Từ lỏng / đồng nghĩa:** kiểm tra khả năng hiểu ngôn ngữ mạng/từ lỏng (*Embedding robust*).
- **Đặc điểm chuyên sâu (IMPACT):** kiểm tra năng lực đọc hiểu văn bản dài của *Cross-Encoder Rerank*.
- **Truy vấn siêu ngắn (“FiiO JD7”):** kiểm tra khả năng đối sánh chính xác (*Exact match*) và thứ tự xếp hạng (*ranking*) với dữ liệu đầu vào cực ít.

3.3 Chỉ số Đánh giá

Bốn chỉ số chuẩn Information Retrieval được tính tại $K = 5$:

$$P@K = \frac{|\{d \in \text{Top-}K : \text{rel}(d)\}|}{K} \quad (3.1)$$

$$HR@K = 1[\exists d \in \text{Top-}K : \text{rel}(d)] \quad (3.2)$$

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q} \quad (3.3)$$

$$nDCG@K = \frac{DCG@K}{IDCG@K}, \quad DCG@K = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)} \quad (3.4)$$

Ngoài ra, có thêm hàm `is_relevant()` xác định tài liệu liên quan theo chiến lược hai cấp:

- So khớp URL chính xác nếu có ground-truth URL.
- So khớp từ khóa trong metadata nếu query là dạng mở (open-ended).

3.4 Kết quả và Phân tích

3.4.1 Kết quả Tổng thể

Bảng 3.1: So sánh hiệu năng hai chế độ retrieval ($K = 5$)

Phương pháp	P@5	HR@5	MRR	nDCG@5
Dense (FAISS)	0.250	0.667	0.496	0.502
Hybrid+Rerank	0.317	0.833	0.792	0.796

3.4.2 Phân tích theo Loại Truy vấn

Exact match (Q1, Q7, Q12): BM25 đóng vai trò quan trọng trong việc khớp chính xác tên model (“Oracle MKIII”, “FiiO JD7”). Hybrid nhỉnh hơn Dense khoảng 10–

15% MRR do BM25 nâng hạng document chứa từ khóa chính xác.

Semantic query (Q6, Q11): Dense Bi-Encoder xử lý tốt, nắm bắt ngữ nghĩa “đồng nguyên chất” $\approx$ “OCC Litz” hay “IMPACT technology” $\approx$ “Oracle MKIII”. Cross-Encoder nâng thêm chất lượng sắp xếp nhờ đọc toàn bộ nội dung review.

Điều kiện lọc giá (Q2, Q8): Cơ chế Hard Filter (lọc cứng) giá hoạt động rất hiệu quả. Các hàm `parse_price_constraint()` và `price_ok()` đã loại bỏ các sản phẩm vượt ngân sách (ví dụ “dưới 1 triệu”) khỏi kết quả cuối cùng, giúp tối ưu hơn so với Dense thuần túy vốn không có cơ chế lọc cấu trúc này.

Sai chính tả / từ lóng (Q4, Q10): Embedding ngữ nghĩa giúp Dense vẫn tìm được “Ignis” khi query viết “Ignit”, hay “đệm pad” khi query dùng “mút tai nghe”. BM25 thất bại hoàn toàn trong trường hợp này nhưng Dense bù đắp sau quá trình fusion.

3.4.3 Ví dụ Kết quả Thực tế

Bảng 3.2: Top-5 kết quả cho truy vấn “ThieAudio Oracle MKIII” với score đã bonus

Hạng	Sản phẩm	Score	score bonus
1	Tai nghe ThieAudio Oracle MKIII	1.000	+0.5
2	Tai nghe ThieAudio Monarch MKIII	0.976	+0.5
3	Tai nghe ThieAudio Oracle MKII	0.883	+0.5
4	Tai nghe ThieAudio Hype 2	0.746	+0.5
5	Tai nghe ThieAudio Elixir	0.740	+0.5

Kết quả trên cho thấy hệ thống xếp đúng sản phẩm mục tiêu ở vị trí số 1. Các vị trí 2 cùng hãng, nhưng khác model sản phẩm, vị trí 3 cũng cùng thương hiệu ThieAudio nhưng sai tên sản phẩm luôn.

CHƯƠNG 4. THẢO LUẬN VÀ KẾT LUẬN

4.1 Điểm mạnh của Hệ thống

Thiết kế module hoá rõ ràng. Các module với vai trò riêng biệt. Việc thay thế từng thành phần như Model khác không ảnh hưởng đến phần còn lại của pipeline.

Deduplication hai lớp. Hàm `dedup_by_url()` áp dụng sau cả bước fusion (Dense + Sparse) lẫn sau Reranking. Giải quyết vấn đề 1 sản phẩm có dữ liệu bị chunking ra thành nhiều chunk và lúc truy vấn sẽ gặp nhiều chunk trùng url sản phẩm.

Xử lý Metadata thông minh. Cơ chế cộng điểm nếu khớp tên hãng (Brand Bonus) tại bước Rerank và Lọc cứng theo giá (Hard Price Filter) giúp hiểu ý định người dùng và trả về kết quả chính xác đối với các truy vấn có điều kiện ngân sách rõ ràng.

4.2 Hạn chế

Chưa tích hợp LLM generation. Hàm `rag_answer()` hiện chỉ trả về danh sách tài liệu truy xuất tốt nhất cùng với placeholder. Phần sinh câu trả lời tự nhiên từ context – vòng lặp RAG thế hệ mới chưa được triển khai hoàn thiện.

Bộ test quy mô nhỏ. Tập kiểm thử hiện tại chưa đủ đại diện thống kê cho phân phối truy vấn thực tế. Độ tin cậy của các chỉ số đánh giá cần được kiểm chứng trên tập câu hỏi lớn hơn.

Phụ thuộc cấu trúc HTML. Crawler sẽ gặp lỗi nếu website thay đổi layout (các class selector như `emtry_content`, `price_config`).

Thiếu query rewriting. Hệ thống chưa có bước paraphrase để mở rộng truy vấn, ảnh hưởng đến độ phủ (recall) đối với các câu query siêu ngắn hoặc quá mơ hồ.

4.3 Kết luận

Đồ án đã xây dựng hệ thống RAG cho tìm kiếm sản phẩm tai nghe tiếng Việt, bao phủ pipeline từ thu thập dữ liệu đến retrieval và đánh giá. Đóng góp chính của dự án là kiến trúc **Hybrid Search**, **Cross-Encoder Reranking** và cơ chế xử lý **Metadata (Brand Bonus & Price Filter)** theo ngữ cảnh.

Kết quả thực nghiệm xác nhận cấu hình Hybrid+Rerank vượt trội so với Dense Search đơn thuần trên tất cả các chỉ số.